

AI-Driven Self-Adaptive Network Digital Twin

Formula Reference

Every formula genuinely implemented and used by this project — fidelity metrics, the deterministic acceptance gate, PPO reinforcement learning, and telemetry-derived features — reproduced exactly from the source code, not a generic textbook restatement.

2026-09-07

Every formula on the following pages is cross-referenced to the exact file/function that implements it — see the appendix on the last page.

Raw Fidelity Metrics (1/2)

Four deterministic metrics compare a component's predictions $\hat{y}$ against ground truth y over an evaluation window of n samples. All four are pure functions — no randomness, no LLM involvement anywhere in this computation (rule 9).

Root Mean Squared Error (pointwise)

$$RMSE(y, \hat{y}) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Mean Absolute Error (pointwise)

$$MAE(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

1-Wasserstein / Earth Mover's Distance (distributional)

Order/pairing-independent — compares y and $\hat{y}$ as empirical distributions. Computed via the closed-form 1-D optimal-transport solution (sorted order statistics), not an approximation:

$$W_1(P, Q) = \int_{-\infty}^{\infty} |F_P(x) - F_Q(x)| dx$$

F_P and F_Q are the empirical CDFs of y and $\hat{y}$ (`scipy.stats.wasserstein_distance`).

Raw Fidelity Metrics (2/2)

MK-MMD — Multi-Kernel Maximum Mean Discrepancy (distributional)

A genuine multi-kernel implementation: the unbiased MMD² estimator is computed at three fixed RBF bandwidths around a base γ , averaged, clamped at 0, then square-rooted.

RBF kernel:

$$k_{\gamma}(a, b) = \exp(-\gamma \|a - b\|^2)$$

Unbiased MMD² estimator (excludes self-pair $i=j$ terms):

$$\widehat{MMD}_u^2 = \frac{1}{n(n-1)} \sum_{i \neq j} k(y_i, y_j) + \frac{1}{m(m-1)} \sum_{i \neq j} k(\hat{y}_i, \hat{y}_j) - \frac{2}{nm} \sum_{i, j} k(y_i, \hat{y}_j)$$

Averaged over three bandwidth multipliers $\{0.5, 1.0, 2.0\} \times \gamma_0$:

$$MK-MMD = \sqrt{\max\left(0, \frac{1}{3} \sum_{c \in \{0.5, 1.0, 2.0\}} \widehat{MMD}_u^2(\gamma = c \gamma_0)\right)}$$

Base bandwidth γ_0 = config value, or the median heuristic when unset (deterministic):

$$\gamma_0 = \frac{1}{2 \cdot \text{median}(\{\|p_i - p_j\|^2 : i \neq j\})}$$

Composite Fidelity Score

Every raw metric is squared, then min-max normalized against a rolling window of historical squared values for that (component, metric) pair — never against the current point alone. All four normalized metrics are then averaged into one composite score.

Step 1 — square every raw metric

$$D_m = (\text{metric}_m)^2 \quad m \in \{RMSE, MAE, W_1, MKMMD\}$$

Step 2 — rolling min-max normalization

$$\tilde{D}_m = \frac{D_m - \min(D_m)}{\max(D_m) - \min(D_m) + \varepsilon}$$

min/max are over the metric's rolling historical window

(`config.fidelity.rolling_window_length`, default 200), evaluated BEFORE the current point is appended. $\varepsilon = \text{config.fidelity.epsilon}$ (default 1e-8) stabilizes a near-constant window.

Step 3 — composite score

$$S_{raw,c} = \tilde{D}_{RMSE} + \tilde{D}_{MAE} + \tilde{D}_{W_1} + \tilde{D}_{MKMMD}$$

$$FidelityScore_c = 1 - \frac{S_{raw,c}}{4}$$

Fewer than `config.fidelity.min_history_for_normalization` (default 10) prior points -> the composite score is withheld (None, status="insufficient_history"), never fabricated.

Worked hand-computed example (this project's own test suite): $y=[0,0]$, $\hat{y}=[a,a]$ for $a=1,2,3$ (point-mass, so $RMSE=MAE=W_1=a$ exactly). For $a=3$: $FidelityScore = -1.287340459401399$, verified to 9 significant figures against an independently hand-derived value.

Deterministic Acceptance Gate

The ONLY criterion that decides ACCEPT vs. REJECT for a candidate model version. Computed by recomputing FidelityScore for both the current production model and the candidate on the identical held-out evaluation window and normalization reference (Module 12).

Primary criterion

$$FidelityScore_{new} > FidelityScore_{old} + \delta$$

δ = config.adaptation.verification_delta (0.01) — never hardcoded elsewhere.

Mandatory conditions (ALL must pass, or REJECT)

- candidate artifact exists on disk and is non-empty
- candidate's interface identity matches (COMPONENT_NAME / OUTPUT_FIELD)
- evaluation genuinely ran (prediction/target array lengths match, non-empty)
- no invalid output (every prediction finite — no NaN/Inf)
- the production baseline was itself evaluable under the same protocol
- the primary fidelity criterion above

Special case — no production baseline exists (a genuinely new component)

The primary criterion is inapplicable (no FidelityScore_old). The gate falls back to requiring only that the candidate's own FidelityScore be well-defined — never fabricating an “old” score to compare against.

Rule 9 — the LLM cannot override this

An optional LLM reasoning step runs strictly AFTER the decision above is already final and already acted on. Its structured-output schema has exactly two fields — explanation, key_observations — genuinely no field anywhere that could express a verdict.

Reinforcement Learning: MDP Formulation

The adaptation-strategy decision is framed as a Markov Decision Process. PPO (Proximal Policy Optimization, Schulman et al. 2017, via Stable-Baselines3) learns a stochastic policy $\pi_\theta(a|s)$ over this MDP; PPO ONLY ever selects the action.

Action space — Discrete(3)

$$a \in \{0 : \text{recalibrate}, 1 : \text{regenerate}, 2 : \text{expand_scope}\}$$

Observation vector (23-dim, all entries clipped to $[-2, 2]$)

$$s = [f_1, \dots, f_5, \text{aff}, \tilde{\sigma}, \text{prevact}, \tilde{r}_{prev}, \mathbf{n}]$$

$f_1..f_5$ = per-component FidelityScore (clipped $[-3,1]$); aff = affected-component one-hot; $\tilde{\sigma}$ = normalized severity; prevact = previous-action one-hot; $\tilde{r}$ = previous reward (clipped); $\mathbf{n}$ = 8-dim network-state summary.

Severity normalization

$$\tilde{\sigma} = \frac{\sigma - \sigma_{lo}}{\max(\sigma_{hi} - \sigma_{lo}, \epsilon)}$$

$[\sigma_{lo}, \sigma_{hi}] = \text{config.drift.severity_range}$ (0.0, 1.0).

Network-state feature normalization (self-referential min-max)

$$n_j = \frac{\bar{x}_j - \min(x_j)}{\max(\max(x_j) - \min(x_j), \epsilon)}$$

x_j = one of 8 real D1 telemetry features, averaged over a sampled window; min/max taken over the same pool/window — identical formula in training (rl_env.py) and live runtime (main.py).

Reward (identical formula in training and live runtime)

$$r = (\text{FidelityScore}_{after} - \text{FidelityScore}_{before}) - c(a)$$

$c(a) = \text{config.ppo.reward.adaptation_cost_penalty}[a]$: recalibrate 0.02, regenerate 0.08, expand_scope 0.12.

The PPO Algorithm (1/2)

This project trains a genuine Stable-Baselines3 PPO model against the MDP on the previous page — never a disguised rule-based function. The standard formulas below are the algorithm SB3 implements; this project configures and trains against them (`config.ppo.training`), it does not re-derive them.

Clipped surrogate objective

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

$$L^{CLIP}(\theta) = E_t[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$

$\epsilon = \text{config.ppo.training.clip_range} = 0.2$.

Generalized Advantage Estimation (GAE)

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

$\gamma = \text{config.ppo.training.gamma} = 0.99$, $\lambda = \text{config.ppo.training.gae_lambda} = 0.95$.

The PPO Algorithm (2/2)

Value function loss + entropy bonus + total loss

$$L^V(\theta) = E_t[(V_\theta(s_t) - V_t^{targ})^2]$$

$$L(\theta) = E_t[L^{CLIP}(\theta) - c_1 L^V(\theta) + c_2 H[\pi_\theta](s_t)]$$

Configured hyperparameters (config.ppo.training)

```
learning_rate = 3.0e-4      n_steps = 256      batch_size = 64
gamma = 0.99               gae_lambda = 0.95    clip_range = 0.2
n_envs = 4                 total_timesteps = 200,000 (default config value; a reduced
                           20,480-step run was actually performed this project – see
                           IMPLEMENTATION_STATUS.md for the real observed reward curve)
```

Runtime inference (the ONLY function that ever selects a strategy)

$$a^* = \arg \max_a \pi_\theta(a | s)$$

(deterministic=True — runtime behavior is reproducible given the same observation.)

A deterministic config fallback (config.ppo.fallback.default_action) is used ONLY on genuine PPO infrastructure failure (missing/corrupt policy file), always logged at WARNING — never a routine substitute for the trained policy.

Simulated Adaptation-Outcome Dynamics

Used ONLY inside AdaptationEnv to give PPO something to train against — a documented “world model” (like CartPole's physics), never a decision-making heuristic. PPO still chooses the action; this only defines how the simulated world reacts.

Per-attempt error-magnitude update

$$\rho_{t+1} = \text{clip}(\rho_t \cdot (1 - \eta_a), 0, 1)$$

ρ = remaining_error_fraction (starts at 1.0); η_a = the chosen action's efficacy (below).

Prediction-error std feeding the next simulated fidelity evaluation

$$\sigma_{err} = \sigma_{healthy} + \kappa \cdot \sigma_{drift} \cdot \rho$$

$\sigma_{healthy}$ = env.healthy_noise_std (0.05), κ = env.drift_error_scale (3.0).

Action efficacy η_a (fraction of remaining error removed per attempt)

$$\eta_{recalibrate} = \eta_{rc}^0 \cdot \max(0, 1 - \sigma_{drift}) \cdot \beta^k$$

η_{rc}^0 =0.65 (recalibrate_efficacy); β =0.35 (attempt_diminishing_factor); k = prior recalibrate attempts on this incident.

$$\eta_{regenerate} = \eta_{rg}^0 \cdot \max(0, 1 - \mu \sigma_{drift})$$

η_{rg}^0 =0.85 (regenerate_efficacy); μ =0.3 (regenerate_severity_sensitivity).

$$\eta_{expand} = \eta_{es}^0 \text{ if } k_{prior} \geq 2 \text{ else } \eta_{es}^0 \cdot \phi$$

η_{es}^0 =0.90 (expand_scope_efficacy); ϕ =0.1 (expand_scope_premature_factor) — only genuinely effective once other strategies have already failed twice.

Telemetry-Derived Formulas

Two real, non-fabricated derived quantities computed from raw simulator/telemetry data — not predicted by any DT model, genuine measured/aggregated inputs.

PRB utilization ratio (NS-3 exporter, nr_5g_telemetry_sim.cc)

Real fraction of the time-frequency PRB grid used for data over one telemetry interval: used RB-symbols (real “RBDDataStats” gNB PHY trace) over available RB-symbols (available RBs × data symbols scheduled, from the real “SlotDataStats” trace).

$$U_{PRB} = \min \left(1, \frac{\sum \text{usedRbSymbols}}{\text{availRb} \times \text{dataSymCount}} \right)$$

Cell load aggregate (PrbUtilizationModel's derived input feature)

A cell's aggregate traffic demand at one instant — sum of every attached UE's offered load at that timestamp. Deliberately NOT approximated by PRB utilization itself (that would be leakage, since PRB utilization is this model's own prediction target).

$$L_{cell}(c, t) = \sum_{i : cell(i) = c, time(i) = t} offered_load_i$$

Formula → Source Code Map

Every formula in this document is implemented at exactly the following location — re-derive nothing from memory; read the cited function for the authoritative version.

RMSE / MAE / Wasserstein / MK-MMD

```
src/fidelity/metrics.py
```

Composite FidelityScore (squaring, rolling min-max, average)

```
src/fidelity/evaluator.py : FidelityEvaluator.evaluate()
```

Deterministic acceptance gate

```
src/adaptation/verification_agent.py : VerificationAgent.verify()
```

PPO action space / reward config

```
config/settings.yaml : ppo.action_mapping, ppo.reward
```

Runtime observation construction (live)

```
src/main.py : ContinuousOrchestrator._build_runtime_observation()
```

Training-time observation construction

```
src/adaptation/rl_env.py : AdaptationEnv._build_observation()
```

Live reward computation

```
src/main.py : ContinuousOrchestrator._run_adaptation_cycle()
```

PPO algorithm (clip objective / GAE / value / entropy)

```
stable_baselines3.PPO, configured by src/adaptation/rl_agent.py : build_ppo_agent()
```

Simulated action-efficacy dynamics (training only)

```
src/adaptation/rl_env.py : AdaptationEnv._action_efficacy()
```

PRB utilization ratio (real NS-3 measurement)

```
ns3_sim/nr_5g_telemetry_sim.cc
```

Cell load aggregate

```
src/dt_models/prb_utilization.py : PrbUtilizationModel._add_cell_load()
```

See CLAUDE.md for the full architecture/design rationale behind each of these, and IMPLEMENTATION_STATUS.md for real, observed numeric results from running them.